

P2636R0

References to ranges should always be viewable

Published Proposal, 2022-09-14

Author:

[Hannes Hauswedell](#)

Audience:

LEWG, SG9

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes that the `views::all` adaptor and the `viewable_range` concept accept lvalue references to move-only views.

Table of Contents

1 Revision History

1.1 Revision 0 - 2022-09-14

2 Introduction

2.1 Minimal problem description

3 Detailed examples

3.1 Views from lvalue reference

3.2 View composition (starting from lvalue reference)

3.3 Views from rvalues

3.4 View composition (starting from rvalue)

3.5 Generic code

4 Proposal

5 "Workarounds"

6 Lifetime

6.1 Returning views on local variables

6.2 Returning views on in-parameters

7 Summary

8 References

§ 1. Revision History

§ 1.1. Revision 0 - 2022-09-14

- Initial version.

§ 2. Introduction

When dealing with views, two concepts are important: `view` ("ranges that are views themselves") and `viewable_range` ("ranges that can be turned into views"). Initially, the requirements for the first were very strict, encompassing only non-owning, semi-regular types. The second concept encompassed "views and any lvalue-references to ranges".

Over time, both concepts were relaxed considerably. Views can now be move-only ([P1456](#)) and owning ([P2415](#)), rvalues of non-views are now viewable—in fact most ranges people encounter are now viewable. However, in one particular case, the concepts are now stricter than before: lvalue-references to certain ranges, move-only views, are not viewable. This means that while almost all non-view ranges can be easily composed with view adaptors, certain views now do not qualify for this.

This issue was briefly discussed on the LWG reflector in 2020 where another member highlighted the wide-ranging impact of this change:

I'm concerned about having components that work with ranges but not views, given that views are supposed to be just ranges with additional properties.

A possible "fix" was mentioned in that discussion:

I can envision an alternate design in which `views::all(E)` does admit move-only lvalue views [...] I think that design change needs LEWG approval [...]

But apparently there was no support for this change or no will to pursue it further. This paper picks up that discussion and argues why such a change would indeed make a lot of sense.

§ 2.1. Minimal problem description

```
void foobar(ranges::forward_range auto && r)
{
    auto v = r | views::take(3);
}
```

In the original C++20 standard and in multiple shipped compiler versions, the above code was valid for all `forward_ranges`. Since P2415, the function template is now unconstrained and will fail for certain inputs.

§ 3. Detailed examples

To illustrate the different behaviours, I have chosen to provide examples that start out with (1) a copyable view, (2) a copyable non-view range, (3) a move-only non-view range, and (4) a move-only view.

As example types for non-copyable ranges, I use `Generator` and `GeneratorView`. `Generator` could refer to a user-provided type that generates objects upon iteration (e.g. records of data from a stream); it is a move-only single-pass input-range.

The type `GeneratorView` is the same as `Generator` except that it also inherits from the empty base class `ranges::view_base`. This has no influence on the semantics of the type other than it begin treated as a view by current standard libraries. While we might suggest to users that such ranges should always be implemented as views, we cannot assume that this will always be the case (there are ranges that are neither containers nor views!).

§ 3.1. Views from lvalue reference

(1)	<pre>string_view s{"foobar"}; auto v = s views::take(3);</pre>	s is <code>decay_copyd</code>
(2)	<pre>string s{"foobar"}; auto v = s views::take(3);</pre>	s is bound by <code>ref_view</code>
(3)	<pre>Generator g{}; auto v = g views::take(3);</pre>	g is bound by <code>ref_view</code>
(4)	<pre>GeneratorView g{}; auto v = g views::take(3);</pre>	currently invalid

Before [P1456](#) and [P2415](#), all of these snippets would have been valid. Currently, (4) is not, because lvalue references to move-only views are not viewable.

This also means that code is rejected by current compilers that was accepted by previous versions of the same compiler (GCC, MSVC). While this is true for other C++20 features, the particular changes to the view concepts were discussed mainly as relaxing constraints and not as introducing new ones.

§ 3.2. View composition (starting from lvalue reference)

(1)	<pre>string_view s{"foobar"}; auto v1 = s views::take(3); auto v2 = v1 views::transform(**/);</pre>	s is <code>decay_copyd</code> v1 is <code>decay_copyd</code>
(2)	<pre>string s{"foobar"}; auto v1 = s views::take(3); auto v2 = v1 views::transform(**/);</pre>	s is bound by <code>ref_view</code> v1 is <code>decay_copyd</code>
(3)	<pre>Generator g{}; auto v1 = g views::take(3); auto v2 = v1 views::transform(**/);</pre>	g is bound by <code>ref_view</code> v1 is <code>decay_copyd</code>
(4)	<pre>GeneratorView g{}; auto v1 = g views::take(3); auto v2 = v1 views::transform(**/);</pre>	currently invalid

(4) is still invalid; the rest are valid.

§ 3.3. Views from rvalues

(1)	<pre>auto v = string_view{"foobar"} views::take(3);</pre>	string_view is <code>decay_copyd</code>
(2)	<pre>auto v = string{"foobar"} views::take(3);</pre>	string is bound by <code>owning_view</code>
(3)	<pre>auto v = Generator{} views::take(3);</pre>	generator is bound by <code>owning_view</code>
(4)	<pre>auto v = GeneratorView{} views::take(3);</pre>	generator is <code>decay_copyd</code> (actually a move)

All of these snippets are currently valid code. In the original ranges proposal, only (1) would have been valid, because the others are rvalues of non-views; (4) would not have been regarded as a view, because it is move-only.

§ 3.4. View composition (starting from rvalue)

(1)	<pre>auto v1 = string_view{"foobar"} views::take(3); auto v2 = v1 views::transform(**/);</pre>	string_view is decay_copyd v1 is decay_copyd
(2)	<pre>auto v1 = string{"foobar"} views::take(3); auto v2 = v1 views::transform(**/);</pre>	string is bound by owning_view line 2 is currently invalid
(3)	<pre>auto v1 = Generator{} views::take(3); auto v2 = v1 views::transform(**/);</pre>	generator is bound by owning_view line 2 is currently invalid
(4)	<pre>auto v1 = GeneratorView{} views::take(3); auto v2 = v1 views::transform(**/);</pre>	generator is decay_copyd (actually a move) line 2 is currently invalid

(2)-(4) are not valid, because `v1` in the respective snippet is move-only. Note, how the valid snippets in this example are different from [those created from lvalues](#). If the `| views::transform(**/)` had been appended to the first line and no `v2` created, it would have worked. This means that—contrary to what many previous ranges guides suggest—it makes an important difference now whether you compose the view incrementally or in one line. And it appears that "creating in one line" needs to be recommended.

§ 3.5. Generic code

Returning to the problem of generic code, assume the following:

(definitions for `to_upper` and `to_lower` omitted)

```
void print_up_low(ranges::forward_range auto && rng)
{
    for (char c : rng | views::transform(to_upper))
        cout << c;
    cout << '\n';
    for (char c : rng | views::transform(to_lower))
        cout << c;
}
```

(1a)	<pre>print_up_low(string_view{"foobar"}); print_up_low(string_view{"foobar"} views::take(3));</pre>	both calls are valid
(2a)	<pre>print_up_low(string{"foobar"}); print_up_low(string{"foobar"} views::take(3));</pre>	first call is valid second call is invalid
(1b)	<pre>string_view s{"foobar"}; print_up_low(s); print_up_low(s views::take(3));</pre>	both calls are valid
(2b)	<pre>string s{"foobar"}; print_up_low(s); print_up_low(s views::take(3));</pre>	both calls are valid

These examples show that it is easy to run into problems with "move-only views", even when just using simple adaptors on types like strings and string_views. Is the behaviour shown above intended? Do we expect developers to understand why 1 of the 8 the calls to `print_up_low()` is invalid and why?

Curiously, it is now the example of "creating a view in one line" that is invalid, while previous examples suggested that exactly this usage pattern increases the likelihood of the code being valid.

§ 4. Proposal

This paper suggests **making all previously shown invalid snippets valid** by changing the `viewable_range` concept and the definition of the `views::all` adaptor.

26.4.5 Other range refinements [range.refinements]

Change the definition of `viewable_range` to:

```
template<class T>
concept viewable_range =
    range<T> &&
    ((view<remove_cvref_t<T>> && constructible_from<remove_cvref_t<T>, T>
    
        (!view<remove_cvref_t<T>> && (is_lvalue_reference_v<T> || (movable<remove
        is_lvalue_reference_v<T> || (movable<remove_reference_t<T>> && !is_initia
```

[26.7.6.1 General [range.all.general]]

Change the definition of `views:all` to

- (2.1) `decay-copy(E)` if that expression is well-formed and the decayed type of `E` models `view`.
- (2.2) Otherwise, `ref_view{E}` if that expression is well-formed.
- (2.3) Otherwise, `owning_view{E}`.

These changes are *breaking* insofar as all concept changes are always potentially breaking. However, this proposal very clearly relaxes the constraints on `viewable_range` so the scope is broadened and in practice more code will work than before and not less. In fact, this will *unbreak* code that was valid in earlier iterations of the standard (and is valid in released versions of GCC and MSVC) but that has been broken in the meantime.

§ 5. "Workarounds"

For the previously shown invalid snippets, it is possible to work around the problem by any or either of the following methods:

1. `move`-ing the initial range into the pipe (resulting in an `decay_copy` or `owning_view`); or
2. manually wrapping the initial range in `ref_view`

Since most users never directly interact with `views::all`, `ref_view` and `owning_view`, I assume that it will be very hard to explain where (and which of) these workarounds are necessary.

Furthermore, this creates a burden on generic code that will either need to `if constexpr` on ranges passed to it by reference or blindly wrap them in `ref_view` to make them viewable.

§ 6. Lifetime

Previous mailing list comments suggested that the changes proposed here would create issues with view lifetime. The following two examples explore some of those implications.

§ 6.1. Returning views on local variables

Nr.	Code	Status quo	This proposal
(1)	<pre>string_view s{"foobar"}; return s views::take(3);</pre>	not dangling	not dangling
(2)	<pre>string s{"foobar"}; return s views::take(3);</pre>	dangling	dangling
(3)	<pre>Generator g{}; return g views::take(3);</pre>	dangling	dangling
(4)	<pre>GeneratorView g{}; return views::take(3);</pre>	invalid	dangling

Returning a view created from a local variable can create a dangling reference. This has always been the case and is inherent to `ranges::ref_view` (and independent of the underlying range). While it could be argued that preventing it for *some ranges* is beneficial, it is unclear why this should only be the case for move-only views and not other ranges; especially since example (2) is likely to appear more often than example (4) and the range in (3) is semantically identical to the range in (4).

On a sidenote: Now that we have move-only owning views, one could make an argument for making all uses of `ranges::ref_view` explicit (removing the option from `views::all` and recommending that users `move` into a view composition by default or explicitly opt into having a reference if desired). However, such a change would be quite disruptive, so pursuing this argument at this point seems futile.

§ 6.2. Returning views on in-parameters

Nr.	Code	Status quo	This proposal
(1)	<pre>auto foo(string_view & s) { auto v1 = s views::take(3); return v1 views::transform(**/); }</pre>	not dangling	not dangling
(2)	<pre>auto foo(string & s) { auto v1 = s views::take(3); return v1 views::transform(**/); }</pre>	not dangling	not dangling
(3)	<pre>auto foo(Generator & g) { auto v1 = g views::take(3); return v1 views::transform(**/); }</pre>	not dangling	not dangling
(4)	<pre>auto foo(GeneratorView & g) { auto v1 = g views::take(3); return v1 views::transform(**/); }</pre>	invalid	valid & not dangling

Example (4) would become valid code through the changes proposed here. Contrary to what one may assume, the example will not return a dangling reference: while `s` is move-only, `v1` will be a

`ref_view` to `s` and thus copyable; in the return statement, it is copied into the returned view, so the composition does not dangle.

§ 7. Summary

Treating move-only views as fundamentally different from copyable views when evaluating view-ability, creates yet another source of user error and frustration. I agree with the changes made to views, but we now have problems explaining what a view even is (P2415); having two sets of views with different usage patterns definitely won't help. This does not only affect exotic, user-defined types, but can become obvious even in toy examples created with strings.

The difference becomes even more pronounced with standalone move-only ranges that can be implemented as views but where we cannot assume that they always will be. For those types, it is unclear what benefit **opting into** the view concept provides—if that only makes the type **opt out** of easy view composition.

Further complications arise in generic code, which could previously treat all views as ranges and now needs special case handling.

While it is true that all direct and indirect uses of `ranges::ref_view` entail the possibility of dangling references, it is unclear why this is considered more dangerous for move-only views than other ranges. I hope to have alleviated some concerns in this regard and shown that the proposed increase in consistency and usability provides a clear benefit.

§ 8. References

- Short 2020 discussion in LWG: <https://lists.isocpp.org/lib/2020/09/17353.php>
- Short 2022 discussion in LWG: <https://lists.isocpp.org/lib/2022/08/23438.php>
- Brief mention in minutes: <https://wiki.edg.com/bin/view/Wg21telecons2021/LWG-issues-20210115>
- GCC issue with other examples: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=106669